

[1]. Tool: ChatGPT

Prompt: Write Python code to extract only the animal data from the CIFAR-10 real data set, given the following instructions from the site and a similar loading method from my previous project.

Output:

Got you — here is the clean, correct version for loading colored CIFAR-10 animal images into X and y.

This will:

Load bird, cat, deer, dog, frog, horse

Keep them in color (RGB, 32×32)

Assign labels 0–5

Produce numpy arrays ready for training

✅ Colored CIFAR-10 Animals Loader (Final Code)

```
import os
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

# Your animal folders
birdpath = "/code/COE379L-Final/cifar10_animals/bird"
catpath = "/code/COE379L-Final/cifar10_animals/cat"
deerpath = "/code/COE379L-Final/cifar10_animals/deer"
dogpath = "/code/COE379L-Final/cifar10_animals/dog"
frogpath = "/code/COE379L-Final/cifar10_animals/frog"
horsepath = "/code/COE379L-Final/cifar10_animals/horse"

# Map classes to labels
paths = {
    "bird": birdpath,
    "cat": catpath,
    "deer": deerpath,
    "dog": dogpath,
    "frog": frogpath,
```

```

    "horse": horsepath
}

label_map = {name: i for i, name in enumerate(paths.keys())}
print("Label mapping:", label_map)

X = []
y = []

# Load colored images
for name, path in paths.items():
    for img_name in os.listdir(path):
        if img_name.lower().endswith((".png", ".jpg", ".jpeg")):
            img_path = os.path.join(path, img_name)

            # Load as RGB, NOT grayscale
            img = load_img(img_path, color_mode="rgb", target_size=(32, 32))

            img_array = img_to_array(img)    # shape: (32, 32, 3)
            X.append(img_array)
            y.append(label_map[name])

# Convert to numpy arrays
X = np.array(X)
y = np.array(y)

print("Loaded X shape:", X.shape)  # → (num_images, 32, 32, 3)
print("Loaded y shape:", y.shape)  # → (num_images,)

```

[2] Tool: ChatGPT

Prompt: does this achieve what i think it should dat[dat['Stage'] == 'test']['Encoded_Class'] = dat[dat['Stage'] == 'test']['Encoded_Class'].apply(lambda x: x - 2)

Output:

```

dat.loc[dat['Stage'] == 'test', 'Encoded_Class'] = (
    dat.loc[dat['Stage'] == 'test', 'Encoded_Class'] - 2
)

```

[3] Tool: Google AI

Prompt: How to shuffle pandas df

Output:

```
import pandas as pd
```

```
# Example DataFrame
```

```
data = {'A': range(1, 6), 'B': ['a', 'b', 'c', 'd', 'e']}
```

```
df = pd.DataFrame(data)
```

```
# Shuffle the DataFrame
```

```
shuffled_df = df.sample(frac=1)
```

```
# Optional: Reset the index to have a clean, sequential 0-based index
```

```
shuffled_df = shuffled_df.reset_index(drop=True)
```

```
print(shuffled_df)
```